



분류 (4) - 캐글 실습

YB 3조 강수민, 김경민, 김나린, 김미소

캐글 산탄데르 고객 만족 예측

- **주제:** 캐글(Kaggle)의 산탄데르 은행 고객 만족도 예측 데이터.
- **목표:** 370개의 익명화된 피처를 활용해 고객이 불만(1)인지 만족(0)인지 분류.

데이터 불균형(대부분 만족, 일부 불만) 때문에 정확도(Accuracy)가 아닌
ROC-AUC를 평가 지표로 사용

1. 데이터 전처리

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
import warnings

warnings.filterwarnings('ignore')
cust_df = pd.read_csv("./train_santander.csv", encoding='latin-1')
print(f'dataset shape: {cust_df.shape}')
cust_df.head(3)
```

- **Dataset Shape:** 76,020 Rows x 371 Columns
- **Feature Types:** float64 111개, int64 260개 => 371개
- **Missing Values:** Null 없음

⇒ 피쳐 대부분이 숫자형이며, 결측치가 거의 없는 정제된 데이터셋

1. 데이터 전처리

```
#Target 속성의 값의 분포
print(cust_df['TARGET'].value_counts())
unsatisfied_cnt = cust_df[cust_df['TARGET'] == 1].TARGET.count()
total_cnt = cust_df.TARGET.count()
print('unsatisfied 비율은 {:.2f}'.format((unsatisfied_cnt / total_cnt)))
```

```
TARGET
0    73012
1     3008
Name: count, dtype: int64
unsatisfied 비율은 0.04
```

⇒ 레이블 값이 극도로 불균형 → ROC-AUC를 평가 지표로 선정

```
cust_df.describe()
```

	ID	var3	var15	imp_ent_var16_ult1	imp_op_var39_comer_ult1
count	76020.000000	76020.000000	76020.000000	76020.000000	76020.000000
mean	75964.050723	-1523.199277	33.212865	86.208265	72.363067
std	43781.947379	39033.462364	12.956486	1614.757313	339.315831
min	1.000000	-999999.000000	5.000000	0.000000	0.000000
25%	38104.750000	2.000000	23.000000	0.000000	0.000000
50%	76043.000000	2.000000	28.000000	0.000000	0.000000
75%	113748.750000	2.000000	40.000000	0.000000	0.000000
max	151838.000000	238.000000	105.000000	210000.000000	12888.030000

8 rows × 371 columns

Min값이 -999,999값으로 나타남
다른 값들에 비해 편차가 너무 커서 모델에 악영
향을 줄 수 있으므로 가장 빈도수가 높은 2로 대체

1. 데이터 전처리

```
#ID 피쳐 드롭
cust_df['var3'].replace(-999999, 2, inplace=True) #이상치 처리
cust_df.drop('ID', axis=1, inplace=True) #피쳐 드롭

# 피쳐 세트와 레이블 세트 분리. 레이블 칼럼은 DataFrame의 맨 마지막에 위치해 칼럼 위치 -1 로 분리
X_features = cust_df.iloc[:, :-1]
y_labels = cust_df.iloc[:, -1]
print('피쳐 데이터 shape: {0}'.format(X_features.shape))
```

피쳐 데이터 shape: (76020, 369)

```
#학습 데이터와 테스트 데이터 세트 추출 확인
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X_features, y_labels, test_size=0.2, random_state=0)

train_cnt = y_train.count()
test_cnt = y_test.count()
print('학습 세트 Shape: {0}, 테스트 세트 Shape: {1}'.format(X_train.shape, X_test.shape))

print('학습 세트 레이블 값 분포 비율')
print(y_train.value_counts() / train_cnt)
print('\n 테스트 세트 레이블 값 분포 비율')
print(y_test.value_counts() / test_cnt)
```

#데이터의 20%를 테스트용으로 분리

학습 세트 Shape: (60816, 369), 테스트 세트 Shape: (15204, 369)

학습 세트 레이블 값 분포 비율

TARGET

0 0.960964

1 0.039036

Name: count, dtype: float64

테스트 세트 레이블 값 분포 비율

TARGET

0 0.9583

1 0.0417

Name: count, dtype: float64

1. 데이터 전처리

```
# X_train, y_train을 다시 학습과 검증 데이터 세트로 분리.  
X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train, test_size=0.3, random_state=0)
```

#데이터 세트 재분할

기존 학습 데이터(X_train)를 다시 7:3 비율로 나누어
최종 학습용(X_tr)과 검증용(X_val) 데이터 세트 생성

⇒ 과적합 방지

2. XGBoost 모델 학습과 하이퍼 파라미터 튜닝

```
from xgboost import XGBClassifier
from sklearn.metrics import roc_auc_score

# n_estimators는 500으로, random state는 예제 수행 시마다 동일 예측 결과를 위해 설정.
xgb_clf = XGBClassifier(n_estimators=500, learning_rate=0.05, random_state=156)

# 성능 평가 지표를 auc로, 조기 중단 파라미터는 100으로 설정하고 학습 수행.
xgb_clf.fit(X_tr, y_tr, early_stopping_rounds=100, eval_metric="auc", eval_set=[(X_tr, y_tr), (X_val, y_val)])

xgb_roc_score = roc_auc_score(y_test, xgb_clf.predict_proba(X_test)[: , 1])
print('ROC AUC: {0:.4f}'.format(xgb_roc_score))
```

- **n_estimators=500**: 최대 반복 횟수 설정
- **learning_rate=0.05**: 안정적인 학습을 위해 비교적 낮은 학습률 채택
- **Early Stopping (100회)**: 검증 오차가 100회 이상 개선되지 않으면 학습을 자동 종료하여 효율성 극대화 및 과적합 방지

2. XGBoost 모델 학습과 하이퍼 파라미터 튜닝

```
from hyperopt import hp

# max_depth는 5에서 15까지 1 간격으로, min_child_weight는 1에서 6까지 1 간격으로
# colsample_bytree는 0.5에서 0.95 사이, learning_rate는 0.01 에서 0.2 사이 정규 분포된 값으로 검색.
xgb_search_space = {
    'max_depth': hp.quniform('max_depth', 5, 15, 1),
    'min_child_weight': hp.quniform('min_child_weight', 1, 6, 1),
    'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 0.95),
    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2)}
```

#HyperOpt 기반 베이지안 최적화

: 단순히 모든 조합을 시도하는 Grid Search보다 효율적으로 최적의 파라미터를 탐색

2. XGBoost 모델 학습과 하이퍼 파라미터 튜닝

```
from sklearn.model_selection import KFold
from sklearn.metrics import roc_auc_score

# fmin()에서 호출 시 search space 값으로 XGBClassifier 교차 검증 학습 후 -1* roc_auc 평균 값을 반환
def objective_func(search_space):
    xgb_clf = XGBClassifier(n_estimators=100,
                           max_depth=int(search_space['max_depth']),
                           min_child_weight=int(search_space['min_child_weight']),
                           colsample_bytree=search_space['colsample_bytree'],
                           learning_rate=search_space['learning_rate'])

    # 3개 k-fold 방식으로 평가된 roc_auc 지표를 담는 list
    roc_auc_list = []

    # 3개 k-fold 방식 적용
    kf = KFold(n_splits=3)
    # X_train을 다시 학습과 검증용 데이터로 분리
    for tr_index, val_index in kf.split(X_train):
        # kf.split(X_train)으로 추출된 학습과 검증 index 값으로 학습과 검증 데이터 세트 분리
        X_tr, y_tr = X_train.iloc[tr_index], y_train.iloc[tr_index]
        X_val, y_val = X_train.iloc[val_index], y_train.iloc[val_index]

        # early stopping은 30회로 설정하고 추출된 학습과 검증 데이터로 XGBClassifier 학습 수행.
        xgb_clf.fit(X_tr, y_tr, early_stopping_rounds=30, eval_metric="auc",
                   eval_set=[(X_tr, y_tr), (X_val, y_val)])

        # 1로 예측한 확률값 추출 후 roc_auc 계산하고 평균 roc_auc 계산을 위해 list에 결과값 담음.
        score = roc_auc_score(y_val, xgb_clf.predict_proba(X_val)[0, 1])
        roc_auc_list.append(score)

    # 3개 k-fold로 계산된 roc_auc 값의 평균값을 반환하되,
    # HyperOpt는 목적함수의 최솟값을 위한 입력값을 찾으므로 -1을 곱한 뒤 반환.
    return -1 * np.mean(roc_auc_list)
```

#목적 함수 설계

3-Fold Stratified K-Fold: 클래스 불균형을 고려하여 각 폴드(Fold)마다 타겟 비율을 유지하며 검증을 수행함.

Early Stopping 적용: 각 폴드 학습 시 30회의 조기 중단을 설정하여 불필요한 연산을 방지하고 과적합을 제어함.

반환값 설정: HyperOpt의 fmin()은 최소값을 찾으므로, ROC-AUC 값에 -1을 곱하여 반환.

2. XGBoost 모델 학습과 하이퍼 파라미터 튜닝

```
from hyperopt import fmin, tpe, Trials

trials = Trials()

# fmin() 함수를 호출, max_evals 지정된 횟수만큼 반복 후 목적함수의 최솟값을 가지는 최적 입력값 추출.
best = fmin(fn=objective_func,
            space=xgb_search_space,
            algo=tpe.suggest, TPE 알고리즘 활용
            max_evals=50, # 최대 반복 횟수를 지정합니다. 50회의 반복 시도
            trials=trials,
            rstate=np.random.default_rng(seed=30))

print('best:', best)
```

#최적화 실행

최적 하이퍼 파라미터 도출 결과

- **colsample_bytree:** 약 0.5749
- **learning_rate:** 약 0.1514
- **max_depth:** 5.0
- **min_child_weight:** 6.0

2. XGBoost 모델 학습과 하이퍼 파라미터 튜닝

```
# n_estimators를 500 증가 후 최적으로 찾은 하이퍼 파라미터를 기반으로 학습과 예측 수행.
xgb_clf = XGBClassifier(n_estimators=500,
                        learning_rate=round(best['learning_rate'], 5),
                        max_depth=int(best['max_depth']),
                        min_child_weight=int(best['min_child_weight']),
                        colsample_bytree=round(best['colsample_bytree'], 5))

# evaluation metric을 auc로, early stopping은 100으로 설정하고 학습 수행.
xgb_clf.fit(X_tr, y_tr, early_stopping_rounds=100,
            eval_metric="auc", eval_set=[(X_tr, y_tr), (X_val, y_val)])

xgb_roc_score = roc_auc_score(y_test, xgb_clf.predict_proba(X_test)[:, 1])
print('ROC AUC: {0:.4f}'.format(xgb_roc_score))
```

최종 성능 비교

Baseline: ROC-AUC 0.8429



After Tuning: ROC-AUC 0.8457

XGBoost는 GBM 대비 빠르고 성능이 우수

그러나,

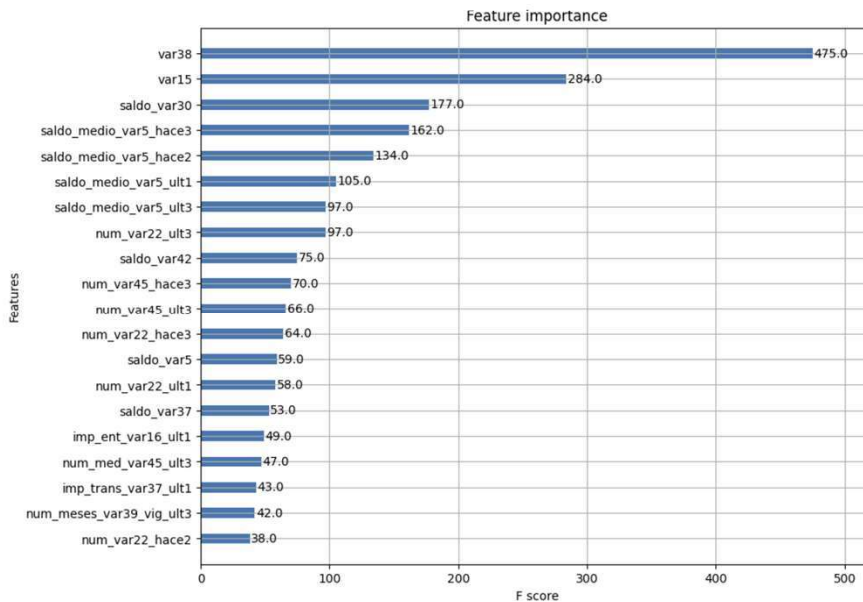
앙상블 알고리즘 특성상 튜닝을 통한 급격한 성능 향상은 어려우며, 수행 시간이 다소 소요됨

2. XGBoost 모델 학습과 하이퍼 파라미터 튜닝

#피쳐 중요도 그래프 나타내기

```
from xgboost import plot_importance
import matplotlib.pyplot as plt
%matplotlib inline
```

```
fig, ax = plt.subplots(1, 1, figsize=(10, 8))
plot_importance(xgb_clf, ax=ax, max_num_features=20, height=0.4)
```



3. LightGBM 모델 학습과 하이퍼 파라미터 튜닝

```
from lightgbm import LGBMClassifier, early_stopping, log_evaluation

lgbm_clf = LGBMClassifier(n_estimators=500)
eval_set = [(X_tr, y_tr), (X_val, y_val)]

lgbm_clf.fit(X_tr, y_tr,
            eval_metric="auc",
            eval_set=eval_set,
            callbacks=[early_stopping(stopping_rounds=100),
                      log_evaluation(period=10)])

lgbm_roc_score = roc_auc_score(y_test, lgbm_clf.predict_proba(X_test)[: , 1])
print('ROC AUC: {0:.4f}'.format(lgbm_roc_score))
```

설정조건

ROC-AUC 약 **0.8384**로 XGBoost 대비 학습 시간이 눈에 띄게 단축됨

#하이퍼 파라미터 검색 공간 설정

```
lgbm_search_space = {
    'num_leaves': hp.quniform('num_leaves', 32, 64, 1),
    'max_depth': hp.quniform('max_depth', 100, 160, 1),
    'min_child_samples': hp.quniform('min_child_samples', 60, 100, 1),
    'subsample': hp.uniform('subsample', 0.7, 1),
    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2)
}
```

3. LightGBM 모델 학습과 하이퍼 파라미터 튜닝

```
#목적 함수 생성
from lightgbm import early_stopping

def objective_func(space):
    # 하이퍼 파라미터 공간에서 값을 받아 모델 생성
    lgbm_clf = LGBMClassifier(
        n_estimators=500,
        num_leaves=int(space['num_leaves']),
        max_depth=int(space['max_depth']),
        min_child_samples=int(space['min_child_samples']),
        learning_rate=space['learning_rate'],
        subsample=space['subsample'],
        # n_jobs=-1 등 필요한 인자 추가
    )

    # 3개 k-fold 방식으로 평가된 roc_auc 지표를 담은 list
    roc_auc_list = []

    # 3개 k-fold 방식 적용
    kf = KFold(n_splits=3)
    # X_train을 다시 학습과 검증용 데이터로 분리
    for tr_index, val_index in kf.split(X_train):
        # kf.split(X_train)으로 추출된 학습과 검증 index 값으로 학습과 검증 데이터 세트 분리
        X_tr, y_tr = X_train.iloc[tr_index], y_train.iloc[tr_index]
        X_val, y_val = X_train.iloc[val_index], y_train.iloc[val_index]

        # early stopping은 30회로 설정하고 추출된 학습과 검증 데이터로 학습 수행.
        lgbm_clf.fit(X_tr, y_tr, callbacks=[early_stopping(stopping_rounds=30)], eval_metric="auc",
                    eval_set=[(X_val, y_val)])

        # 1로 예측한 확률값 추출 후 roc auc 계산하고 평균 roc auc 계산을 위해 list에 곱셈값 담음.
        score = roc_auc_score(y_val, lgbm_clf.predict_proba(X_val)[0, 1])
        roc_auc_list.append(score)

    # 3개 k-fold로 계산된 roc_auc 값의 평균값을 반환하되,
    # HyperOpt는 목적함수의 최솟값을 위한 입력값을 찾으므로 -1을 곱한 뒤 반환.
    return -1 * np.mean(roc_auc_list)
```

#목적 함수 설계

교차 검증: 3-Fold Stratified K-Fold를 적용하여 레이블 불균형을 고려한 학습 수행.

3. LightGBM 모델 학습과 하이퍼 파라미터 튜닝

```
#최적 하이퍼 파라미터 도출
from hyperopt import fmin, tpe, Trials
import numpy as np

trials = Trials()

# rstate 설정을 np.random.RandomState(seed) 형식으로 변경
best = fmin(fn=objective_func,
            space=lgbm_search_space,
            algo=tpe.suggest,
            max_evals=50,
            trials=trials,
            rstate=np.random.default_rng(30)) # default_rng 안에 바로 숫자를 넣어보세요.

print('best:', best)
```

최적화 수행: HyperOpt의 fmin() 함수를 사용하여 max_evals=50회 반복 탐색

3. LightGBM 모델 학습과 하이퍼 파라미터 튜닝

```
#ROC-AUC 평가
lgbm_clf = LGBMClassifier(n_estimators=500,
                          num_leaves=int(best['num_leaves']),
                          max_depth=int(best['max_depth']),
                          min_child_samples=int(best['min_child_samples']),
                          subsample=round(best['subsample'], 5),
                          learning_rate=round(best['learning_rate'], 5))

# evaluation metric을 auc로, early stopping은 100으로 설정하고 학습 수행.
lgbm_clf.fit(X_tr, y_tr, callbacks=[early_stopping(stopping_rounds=100)],
            eval_metric="auc", eval_set=[(X_tr, y_tr), (X_val, y_val)])

lgbm_roc_score = roc_auc_score(y_test, lgbm_clf.predict_proba(X_test)[: , 1])
print('ROC AUC: {0:.4f}'.format(lgbm_roc_score))
```

최종 결과: ROC-AUC 약 0.8446 기록.

모델 평가: XGBoost와 유사한 높은 수준의 예측 성능을 유지하면서도 상대적으로 빠른 학습 시간을 증명함



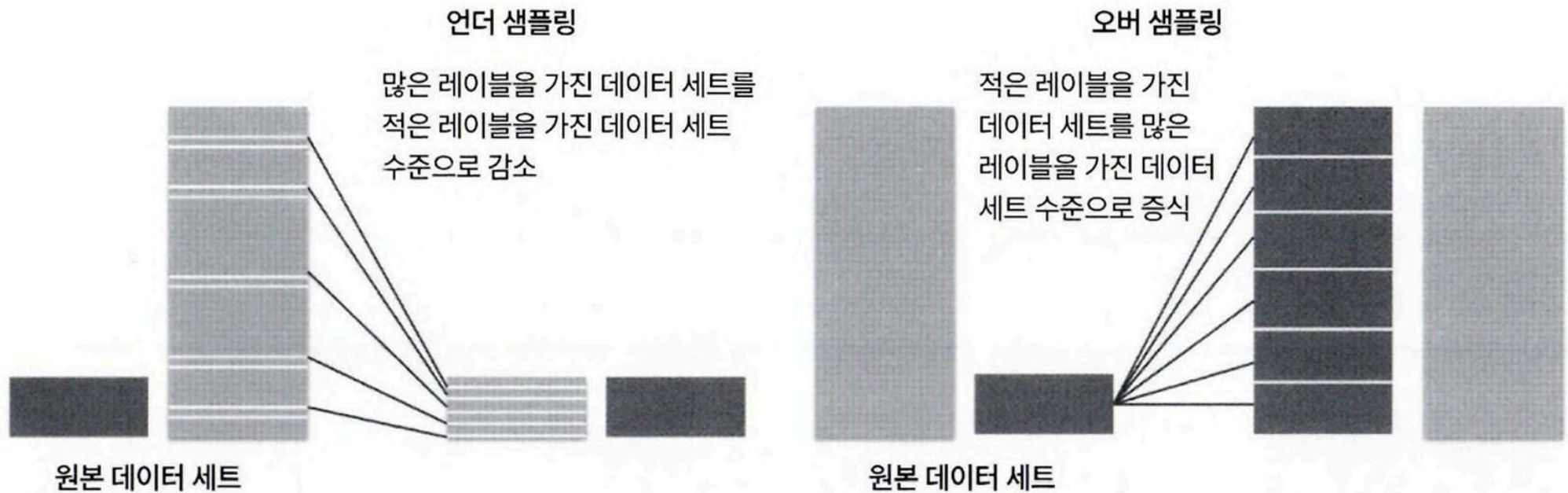
분류 실습 - 캐글 신용카드 사기 검출

데이터의 레이블 분포

- 데이터 세트의 레이블인 Class 속성은 매우 불균형한 분포를 가지고 있음
- 0 : 정상적인 신용카드 트랜잭션 데이터 , 1 : 사기 트랜잭션
- 전체 데이터 중 사기 거래(Class=1)는 약 0.172%에 불과

레이블 불균형 해결방법  언더 샘플링 , 오버 샘플링

언더 샘플링과 오버 샘플링의 이해

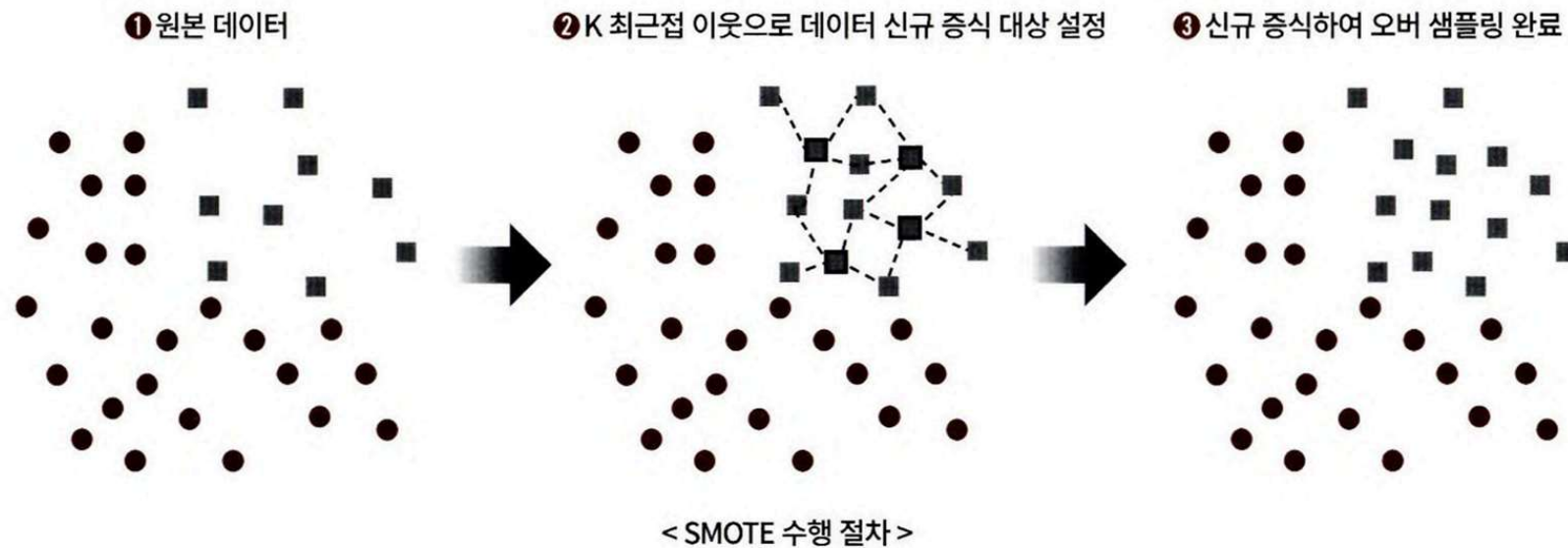


- 언더 샘플링 -> 너무 많은 정상 레이블 데이터를 감소시켜서 정상 레이블의 경우 제대로 된 학습을 수행할 수 없는 문제가 발생할 수도 있음
- 오버 샘플링 -> 대표적으로 SMOTE(Synthetic Minority Over-sampling Technique) 방법이 있음

언더 샘플링과 오버 샘플링의 이해

SMOTE(Synthetic Minority Over-sampling Technique)

: 적은 데이터 세트에 있는 개별 데이터들의 K 최근접 이웃(K Nearest Neighbor)을 찾아서 이 데이터와 K개 이웃들의 차이를 일정 값으로 만들어서 기존 데이터와 약간 차이가 나는 새로운 데이터들을 생성하는 방식



데이터 일차 가공 및 모델 학습/예측/평가

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import warnings
5 warnings.filterwarnings("ignore")
6 %matplotlib inline
7
8 card_df = pd.read_csv('/content/drive/MyDrive/YB_data/creditcard.csv')
9 card_df.head(3)
```

Time 피쳐 제거

Amount : 신용카드 트랜잭션 금액

Class : 0 (정상), 1 (사기)

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...

3 rows × 31 columns

	V21	V22	V23	V24	V25	V26	V27	V28	Amount	Class
	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.189115	0.133558	-0.021053	149.62	0
	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.125895	-0.008983	0.014724	2.69	0
	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.139097	-0.055353	-0.059752	378.66	0

데이터 일차 가공 및 모델 학습/예측/평가

```
1 from sklearn.model_selection import train_test_split
```

```
2
```

```
3 # 인자로 입력받은 DataFrame을 복사한 뒤 Time 칼럼만 삭제하고 복사된 DataFrame 반환
```

```
4 def get_preprocessed_df(df=None):
```

```
5     df_copy = df .copy()
```

```
6     df_copy.drop('Time', axis=1, inplace=True)
```

```
7     return df_copy
```

- `get_preprocessed_df()` : DataFrame 가공하여 반환

- `get_train_test_df()` : 학습/테스트 데이터 세트를 반환

```
1 # 사전 데이터 가공 후 학습과 테스트 데이터 세트를 반환하는 함수.
```

```
2 def get_train_test_dataset(df=None):
```

```
3     # 인자로 입력된 DataFrame의 사전 데이터 가공이 완료된 복사 DataFrame 반환
```

```
4     df_copy = get_preprocessed_df(df)
```

```
5     # DataFrame의 맨 마지막 칼럼이 레이블, 나머지는 피쳐들
```

```
6     X_features = df_copy.iloc[:, :-1]
```

```
7     y_target = df_copy.iloc[:, -1]
```

```
8     # train_test_split()으로 학습과 테스트 데이터 분할, stratify=y_target으로 Stratified 기반 분할
```

```
9     X_train, X_test, y_train, y_test = #
```

```
10     train_test_split(X_features, y_target, test_size=0.3,
```

```
11     | | | | | | | random_state=0, stratify=y_target)
```

```
12     # 학습과 테스트 데이터 세트 반환
```

```
13     return X_train, X_test, y_train, y_test
```

```
14
```

```
15 X_train, X_test, y_train, y_test = get_train_test_dataset(card_df)
```

데이터 일차 가공 및 모델 학습/예측/평가

```
1 print('학습 데이터 레이블 값 비율')
2 print(y_train.value_counts()/y_train.shape[0] * 100)
3 print('테스트 데이터 레이블 값 비율')
4 print(y_test.value_counts()/y_test.shape[0] * 100)
```

학습 데이터 레이블 값 비율

Class

0 99.827451

1 0.172549

Name: count, dtype: float64

테스트 데이터 레이블 값 비율

Class

0 99.826785

1 0.173215

Name: count, dtype: float64

데이터 일차 가공 및 모델 학습/예측/평가

```
1 from sklearn.linear_model import LogisticRegression
2
3 lr_clf = LogisticRegression(max_iter=1000)
4 lr_clf.fit(X_train, y_train)
5 lr_pred = lr_clf.predict(X_test)
6 lr_pred_proba = lr_clf.predict_proba(X_test)[:, 1]
7
8 # 3장에서 사용한 get_clf_eval() 함수를 이용해 평가 수행.
9 get_clf_eval(y_test, lr_pred, lr_pred_proba)
```

오차 행렬

```
[[85281    14]
 [    57    91]]
```

정확도: 0.9992, 정밀도: 0.8667, 재현율: 0.6149, F1: 0.7194, AUC:0.9741

```
1 # 인자로 사이킷런의 Estimator 객체와 학습/테스트 데이터 세트를 입력받아서 학습/예측/평가 수행.
2 def get_model_train_eval(model, ftr_train=None, ftr_test=None, tgt_train=None, tgt_test=None):
3     model.fit(ftr_train, tgt_train)
4     pred = model.predict(ftr_test)
5     pred_proba = model.predict_proba(ftr_test)[:, 1]
6     get_clf_eval(tgt_test, pred, pred_proba)
```

`get_model_train_eval()` : 인자로 사이킷런의 Estimator 객체와 학습/테스트 데이터 세트를 입력받아서 학습/예측/평가를 수행

데이터 일차 가공 및 모델 학습/예측/평가

```
1 from lightgbm import LGBMClassifier
2
3 lgbm_clf = LGBMClassifier(n_estimators=1000, num_leaves=64, n_jobs=-1, boost_from_average=False)
4 get_model_train_eval(lgbm_clf, ftr_train=X_train, ftr_test=X_test, tgt_train=y_train, tgt_test=y_test)
```

레이블 값이 극도로 불균형한 분포를 이루는 경우

오차 행렬

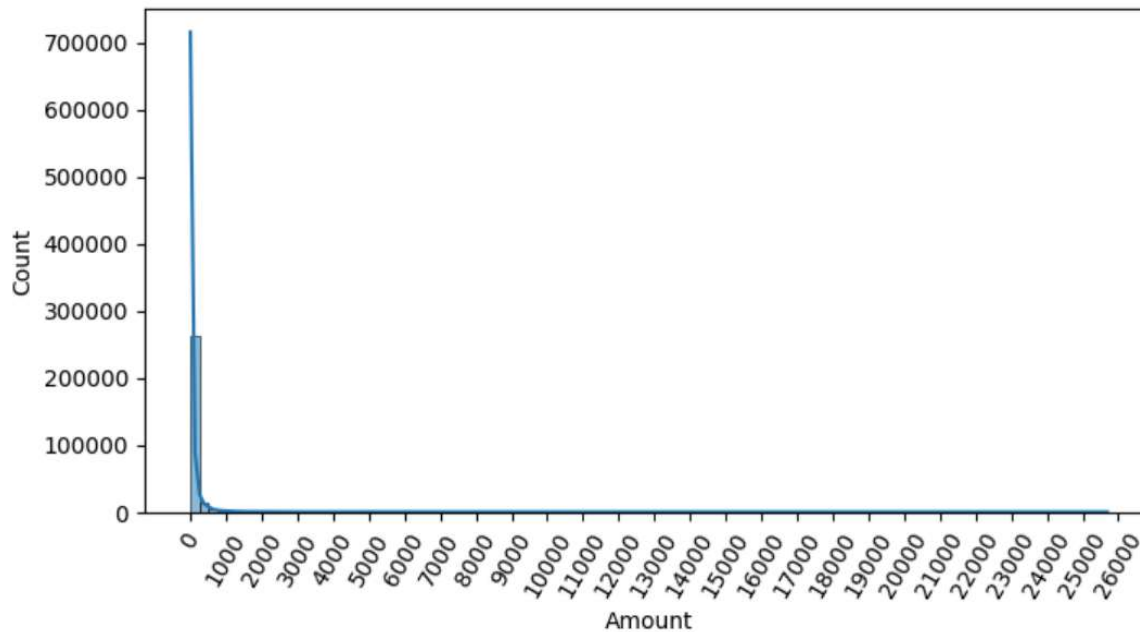
```
[[85290    5]
 [   36   112]]
```

정확도: 0.9995, 정밀도: 0.9573, 재현율: 0.7568, F1: 0.8453, AUC:0.9790

로지스틱 회귀보다 높은 수치

데이터 분포도 변환 후 모델 학습/예측/평가

```
1 import seaborn as sns
2
3 plt.figure(figsize=(8, 4))
4 plt.xticks(range(0, 30000, 1000), rotation=60)
5 sns.histplot(card_df['Amount'], bins=100, kde=True)
6 plt.show()
```



Amount

- 1000불 이하인 데이터가 대부분
- 꼬리가 긴 형태의 분포 곡선

데이터 분포도 변환 후 모델 학습/예측/평가

사이킷런의 StandardScaler 클래스를 이용해
Amount 피처를 정규 분포 형태로 변환하는 코드로 변경

```
1 from sklearn.preprocessing import StandardScaler
2 # 사이킷런의 StandardScaler를 이용해 정규 분포 형태로 Amount 피처값 변환하는 로직으로 수정.
3 def get_preprocessed_df(df=None):
4     df_copy = df.copy()
5     scaler = StandardScaler()
6     amount_n = scaler.fit_transform(df_copy['Amount'].values.reshape(-1, 1))
7     # 변환된 Amount를 Amount_Scaled로 피처명 변경 후 DataFrame맨 앞 칼럼으로 입력
8     df_copy.insert(0, 'Amount-Scaled', amount_n)
9     # 기존 Time, Amount 피처 삭제
10    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)
11    return df_copy
```

데이터 분포도 변환 후 모델 학습/예측/평가

```
1 # Amount를 정규 분포 형태로 변환 후 로지스틱 회귀 및 LightGBM 수행.
2 X_train, X_test, y_train, y_test = get_train_test_dataset(card_df)
3
4 print('### 로지스틱 회귀 예측 성능 ###')
5 lr_clf = LogisticRegression(max_iter=1000)
6 get_model_train_eval(lr_clf, ftr_train=X_train, ftr_test=X_test, tgt_train=y_train, tgt_test=y_test)
7
8 print('### LightGBM 예측 성능 ###')
9 lgbm_clf = LGBMClassifier(n_estimators=1000, num_leaves=64, n_jobs=-1, boost_from_average=False)
10 get_model_train_eval(lgbm_clf, ftr_train=X_train, ftr_test=X_test, tgt_train=y_train, tgt_test=y_test)
```

로지스틱 회귀 예측 성능

오차 행렬

```
[[85281    14]
 [   55    93]]
```

정확도: 0.9992, 정밀도: 0.8692, 재현율: 0.6284, F1: 0.7294, AUC:0.9706

LightGBM 예측 성능

오차 행렬

```
[[85290     5]
 [   37   111]]
```

정확도: 0.9995, 정밀도: 0.9569, 재현율: 0.7500, F1: 0.8409, AUC:0.9779

로지스틱 회귀

- 정밀도와 재현율 조금 저하

LightGBM

- 약간 정밀도와 재현율이 저하되었지만
큰 성능상의 변경 없음

데이터 분포도 변환 후 모델 학습/예측/평가

로그 변환 수행

- 로그 변환 : 데이터 분포도가 심하게 왜곡되어 있을 경우 적용하는 중요 기법 중 하나
- 넘파이의 `log1p()` 함수를 이용

```
1 def get_preprocessed_df(df=None):
2     df_copy = df.copy()
3     # 넘파이의 log1p()를 이용해 Amount를 로그 변환
4     amount_n = np.log1p(df_copy['Amount'])
5     df_copy.insert(0, 'Amount_Scaled', amount_n)
6     df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)
7     return df_copy
```

데이터 분포도 변환 후 모델 학습/예측/평가

```
1 X_train, X_test, y_train, y_test = get_train_test_dataset(card_df)
2
3 print('### 로지스틱 회귀 예측 성능 ###')
4 get_model_train_eval(lr_clf, ftr_train=X_train, ftr_test=X_test,
5 | | | | | | | | | | tgt_train=y_train, tgt_test=y_test)
6
7 print('### LightGBM 예측 성능 ###')
8 get_model_train_eval(lgbm_clf, ftr_train=X_train, ftr_test=X_test,
9 | | | | | | | | | | tgt_train=y_train, tgt_test=y_test)
```

로지스틱 회귀 예측 성능

오차 행렬

```
[[85282    13]
 [    59    89]]
```

정확도: 0.9992, 정밀도: 0.8725, 재현율: 0.6014, F1: 0.7120, AUC:0.9734

LightGBM 예측 성능

오차 행렬

```
[[85290     5]
 [    35   113]]
```

정확도: 0.9995, 정밀도: 0.9576, 재현율: 0.7635, F1: 0.8496, AUC:0.9796

로지스틱 회귀

- 정밀도 향상, 재현율 저하

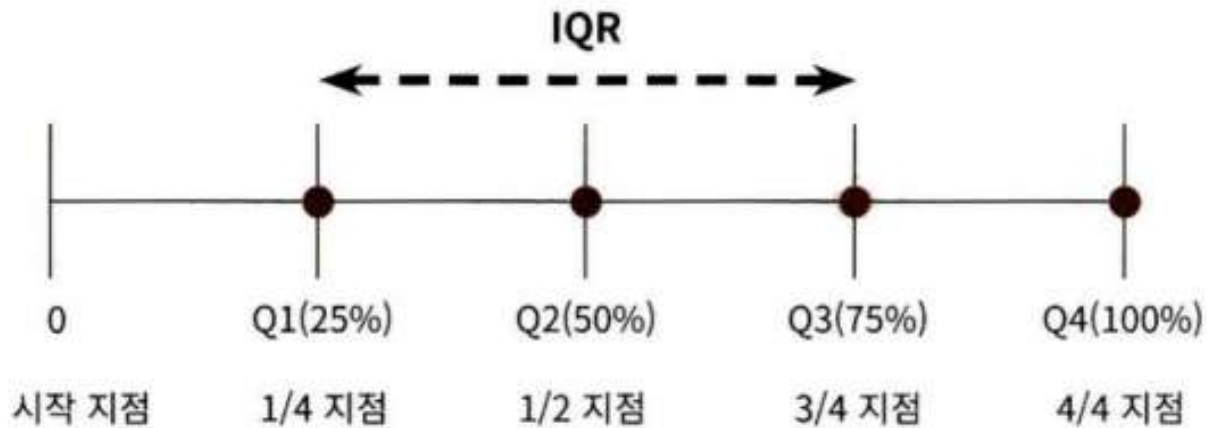
LightGBM

- 재현율 향상

이상치 데이터 제거 후 모델 학습/예측/평가

이상치 데이터(Outlier): 전체 데이터의 패턴에서 벗어난 이상 값을 가진 데이터

IQR(Inter Quantile Range) 방식



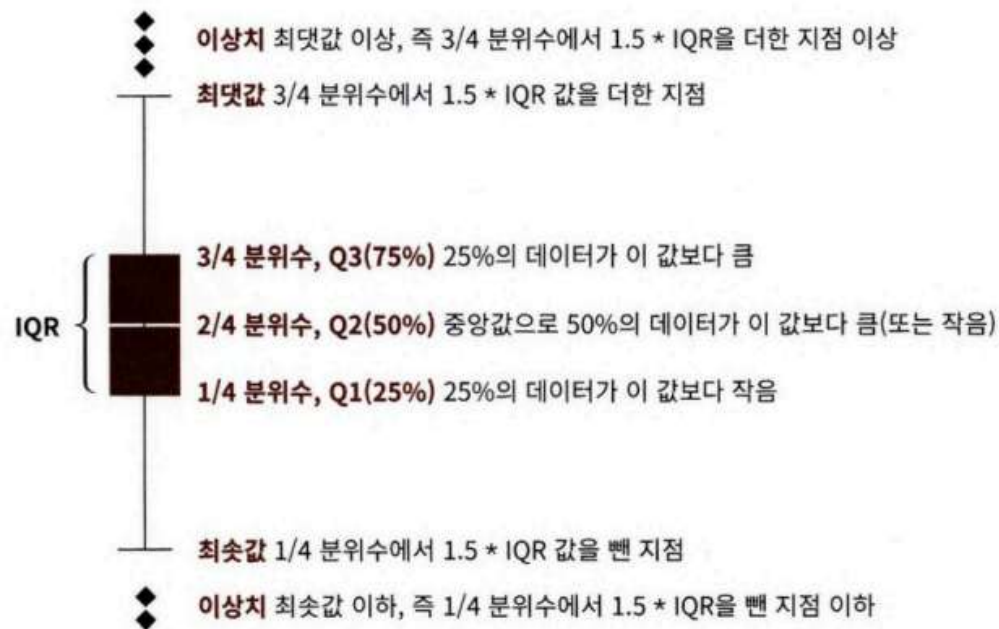
: 사분위(Quantile) 값의 편차를 이용하는 기법 → 흔히 박스 플롯 방식으로 시각화

이상치 데이터 제거 후 모델 학습/예측/평가

$$\text{최댓값} = Q3 + 1.5 * IQR$$

$$\text{최솟값} = Q1 - 1.5 * IQR$$

결정된 최댓값보다 큰 값 또는 최솟값보다 작은 값을 이상치 데이터로 간주



← IQR 방식을 시각화
[박스 플롯]

이상치 데이터 제거 후 모델 학습/예측/평가

결정값과 가장 상관성이 높은 피쳐들을 위주로 이상치를 검출

```
import seaborn as sns
```

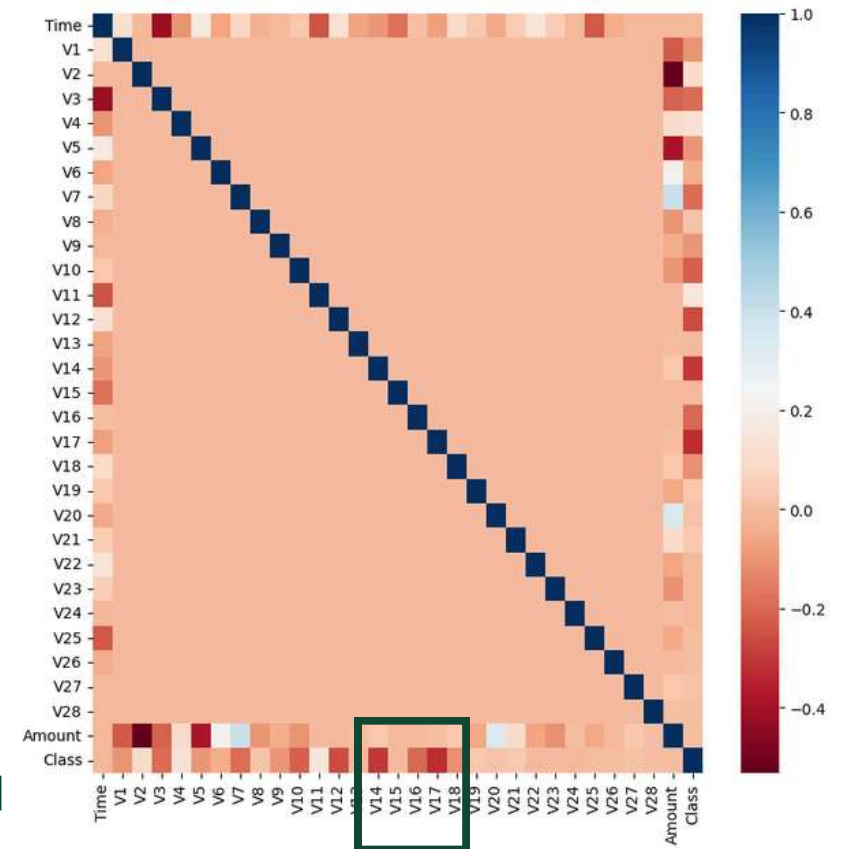
```
plt.figure(figsize=(9,9))
```

```
corr = card_df.corr() ← corr() : 각 피쳐별 상관도 구하는 함수
```

```
sns.heatmap(corr, cmap='RdBu')
```

음의 상관관계가 높을수록 → 진한 빨간색

class피쳐와 음의 상관관계가 가장 높은 V14,V17에 대해서 이상치를 찾아 제거



이상치 데이터 제거 후 모델 학습/예측/평가

```
import numpy as np

def get_outlier(df=None, column=None, weight=1.5):
    # fraud에 해당하는 column 데이터만 추출, 1/4분위와 3/4분위 지점을 np.percentile로 구함
    fraud = df[df['Class']==1][column]
    quantile_25 = np.percentile(fraud.values,25)
    quantile_75 = np.percentile(fraud.values,75)

    #IQR을 구하고 IQR에 1.5를 곱해 최댓값과 최솟값 지점을 구함
    iqr = quantile_75 - quantile_25
    iqr_weight = iqr*weight
    lowest_val = quantile_25 - iqr_weight
    highest_val = quantile_75 + iqr_weight

    #최댓값보다 크거나 최솟값보다 작은 값을 이상치 데이터로 설정하고 DataFrame index 반환
    outlier_index = fraud[(fraud<lowest_val)| (fraud>highest_val)].index
    return outlier_index
```

```
outlier_index = get_outlier(df=card_df, column='V14',weight=1.5)
print('이상치 데이터 인덱스:', outlier_index)
```

```
이상치 데이터 인덱스: Index([8296, 8615, 9035, 9252], dtype='int64')
```

이상치 데이터 제거 후 모델 학습/예측/평가

```
# get_processed_df()을 로그 변환 후 V14피치의 이상치 데이터를 삭제하는 로직으로 변경
def get_preprocessed_df(df=None):
    df_copy = df.copy()
    amount_n = np.log1p(df_copy['Amount'])
    df_copy.insert(0, 'Amount_Scaled', amount_n)
    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)

    #이상치 데이터 삭제하는 로직 추가
    outlier_index = get_outlier(df=df_copy, column='V14', weight=1.5)
    df_copy.drop(outlier_index, axis=0, inplace=True)
    return df_copy

X_train, X_test, y_train, y_test = get_train_test_dataset(card_df)
print('### 로지스틱 회귀 예측 성능 ###')
get_model_train_eval(lr_clf, ftr_train=X_train, ftr_test=X_test, tgt_train=y_train, tgt_test=y_test)

print('### LightGBM 예측 성능 ###')
get_model_train_eval(lgbm_clf, ftr_train=X_train, ftr_test=X_test, tgt_train=y_train, tgt_test=y_test)
```

이상치 추출 & 삭제

로지스틱 회귀 예측 성능

오차 행렬

```
[[85280    15]
 [   48    98]]
```

정확도:0.9993, 정밀도:0.8673, 재현율:0.6712, F1:0.7568, AUC:0.9725

LightGBM 예측 성능

오차 행렬

```
[[85290     5]
 [   25   121]]
```

정확도:0.9996, 정밀도:0.9603, 재현율:0.8288, F1:0.8897, AUC:0.9780

SMOTE 오버 샘플링 적용 후 모델 학습/예측/평가

```
from imblearn.over_sampling import SMOTE
```

SMOTE 구현 패키지

```
smote = SMOTE(random_state=0)
```

SMOTE를 적용할 때는 반드시 학습데이터만 오버샘플링

```
X_train_over, y_train_over = smote.fit_resample(X_train, y_train)
```

```
print('SMOTE 적용 전 학습용 피쳐/레이블 데이터 세트:', X_train.shape, y_train.shape)
```

```
print('SMOTE 적용 후 학습용 피쳐/레이블 데이터 세트:', X_train_over.shape, y_train_over.shape)
```

```
print('SMOTE 적용 후 레이블 값 분포: \n', pd.Series(y_train_over).value_counts())
```

```
SMOTE 적용 전 학습용 피쳐/레이블 데이터 세트: (199362, 29) (199362,)
```

```
SMOTE 적용 후 학습용 피쳐/레이블 데이터 세트: (398040, 29) (398040,)
```

```
SMOTE 적용 후 레이블 값 분포:
```

```
Class
```

```
0    199020
```

```
1    199020
```

```
Name: count, dtype: int64
```

※검증 데이터 세트나 테스트 데이터 세트는 오버샘플링하면 안 됨

SMOTE 오버 샘플링 적용 후 모델 학습/예측/평가 (로지스틱회귀)

```
lr_clf = LogisticRegression(max_iter=1000)
#ftr_train과 tgt_train 인자값이 SMOTE 증식된 X_train_over와 y_train_over로 변경됨에 유의
get_model_train_eval(lr_clf, ftr_train = X_train_over, ftr_test = X_test, tgt_train=y_train_over, tgt_test=y_test)
```

오차 행렬

```
[[82933 2362]
```

```
 [ 11 135]]
```

정확도:0.9722, 정밀도:0.0541, 재현율:0.9247, F1:0.1022, AUC:0.9736

문제점

실제 원본 데이터의 유형보다 너무나 많은 Class=1 데이터를 학습하면서
실제 테스트 데이터 세트에서 Class=1로 예측하는 경우가 많아짐

→ 정밀도 급하락

SMOTE 오버 샘플링 적용 후 모델 학습/예측/평가

```
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
from sklearn.metrics import precision_recall_curve
import numpy as np # Import numpy
%matplotlib inline

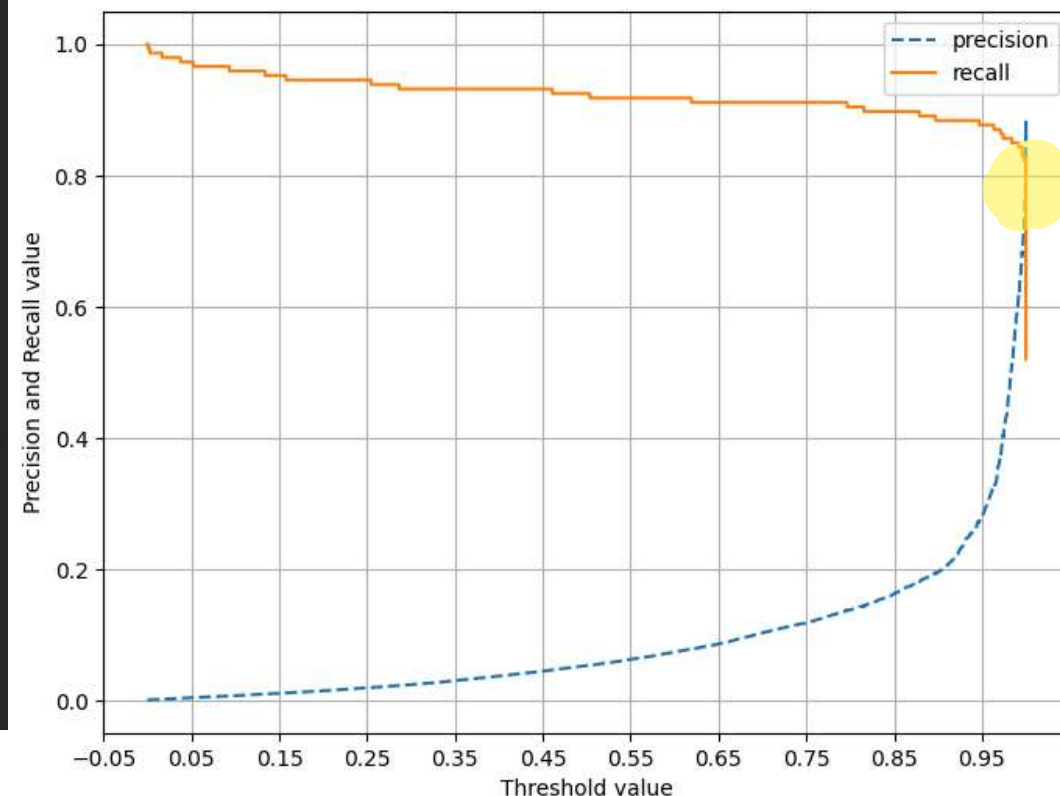
def precision_recall_curve_plot(y_test, pred_proba_cl):
    #threshold ndarray와 이 threshold에 따른 정밀도, 재현율 ndarray 추출
    precisions, recalls, thresholds = precision_recall_curve(y_test, pred_proba_cl)

    # X축을 threshold값으로 Y축은 정밀도, 재현율 값으로 각각 plot수행, 정밀도는 점선으로 표시
    plt.figure(figsize=(8,6))
    threshold_boundary = thresholds.shape[0]
    plt.plot(thresholds, precisions[0:threshold_boundary], linestyle='--', label='precision')
    plt.plot(thresholds, recalls[0:threshold_boundary], label='recall') # Corrected 'recall' to 'recalls'

    # threshold값 X축의 Scale을 0.1단위로 변경
    start, end = plt.xlim()
    plt.xticks(np.round(np.arange(start, end, 0.1),2))

    # x축, y축 label과 legend, 그리고 grid 설정
    plt.xlabel('Threshold value'); plt.ylabel('Precision and Recall value')
    plt.legend(); plt.grid()
    plt.show()

precision_recall_curve_plot(y_test, lr_clf.predict_proba(X_test)[: , 1])
```



SMOTE 오버 샘플링 적용 후 모델 학습/예측/평가 (LightGBM 모델)

```
lgbm_clf = LGBMClassifier(n_estimators=1000, num_leaves=64, n_jobs=-1, boost_from_average=False)
get_model_train_eval(lgbm_clf, ftr_train=X_train_over, ftr_test=X_test, tgt_train=y_train_over, tgt_test=y_test)
```

오차 행렬

```
[[85283  12]
 [   22 124]]
```

정확도:0.9996, 정밀도:0.9118, 재현율:0.8493 F1:0.8794, AUC:0.9814

데이터 가공 유형	머신러닝 알고리즘	평가 지표		
		정밀도	재현율	ROC-AUC
원본 데이터 가공 없음	로지스틱 회귀	0.8679	0.6216	0.9702
	LightGBM	0.9573	0.7568	0.9790
데이터 로그 변환	로지스틱 회귀	0.8812	0.6014	0.9727
	LightGBM	0.9576	0.7635	0.9796
이상치 데이터 제거	로지스틱 회귀	0.8750	0.6712	0.9743
	LightGBM	0.9603	0.8288	0.9780
SMOTE 오버 샘플링	로지스틱 회귀	0.0542	0.9247	0.9737
	LightGBM	0.9118	0.8493	0.9814